VA CLAIMS

Backend Architecture & Implementation Plan

Hackathon HLD + LLD | Frontend / backend alignment | Draft

Executive Summary

Mission

Turn raw veteran records and VA data into a structured, cited, rule-aware case brief that an accredited Veterans Service Officer (VSO) can review in minutes instead of manually researching an entire record.

Design stance

The backend is an evidence-preparation and decision-support system, not an autonomous adjudicator. The software performs the mechanical “dig”; deterministic code evaluates rules that can be computed; the LLM organizes and explains the evidence; the VSO remains the human decision-maker.

What the backend must do

• INGEST — accept authorized VA/API data and veteran-uploaded documents such as medical records, DD-214s, DBQs, prior decisions, and supporting PDFs.

• REASON — normalize the evidence, retrieve what is relevant to the veteran’s query, run deterministic VA-rule checks, and synthesize a source-backed case analysis.

• SERVE — return stable, structured review items that the frontend can render as Confirm / Reject / Needs Review actions with direct evidence references.

MVP guardrails

• No final approve/deny decision by the AI.

• Deterministic facts and rule results come from parsers and rules engines, not free-form model prose.

• Every material finding must resolve to a real source document, page/section, or API record.

• Model output is schema-constrained; hallucinated evidence IDs are rejected before the frontend sees them.

• Minimize retained PII and avoid becoming a second eFolder or medical-record system.

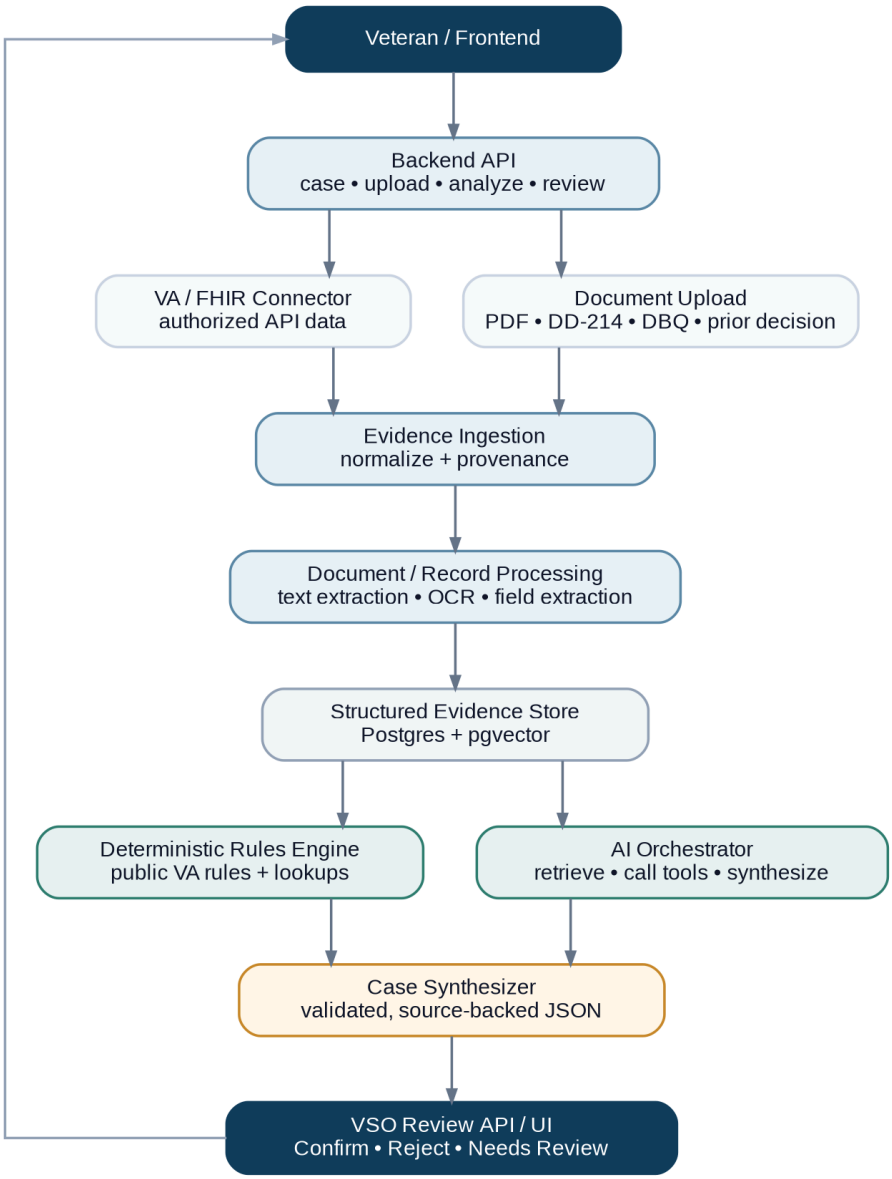
• For the hackathon, prefer uploads plus VA sandbox/mock data over blocking on production access.

Definition of done

A veteran record is uploaded or fetched, the backend extracts and retrieves the relevant facts, deterministic rules run where applicable, the AI produces a validated case brief, and the review UI lets a VSO verify findings without manually searching the entire record.

1. Backend HLD

System architecture



Component responsibilities

Backend API — Single entry point for case creation, uploads, analysis requests, status, and review actions.

Evidence Ingestion — Normalizes records from VA/FHIR connectors and uploads into one source model. Every source receives a case ID, document ID, source type, and provenance metadata before analysis.

Document / Record Processing — Extracts text and structured fields. MVP extraction focuses on diagnoses/conditions, dates, treatment events, medications, service dates, MOS/job code, deployments, medals, prior claim outcomes, and page/record references. OCR is used only when text extraction is unavailable.

Structured Evidence Store — PostgreSQL is the source of truth for normalized evidence and provenance. pgvector provides semantic retrieval without introducing a separate vector database for the hackathon.

Deterministic Rules Engine — Evaluates facts that should be computed rather than guessed, such as public lookup-table matches, service/date windows, and selected presumptive-eligibility rules. Returns structured RuleResult objects with rule ID, inputs, result, and explanation.

AI Orchestrator — Retrieves evidence, invokes deterministic tools, and asks the model to synthesize—not invent—the result. Its job is relevance, organization, explanation, and generation of review items.

Case Synthesizer / Review API — Validates model references, joins evidence and rule results, and returns the frontend contract used to render VSO review cards.

Key implementation choice

The model is not trained to learn "what claims usually win." The MVP uses a foundation model + retrieval + deterministic tools. Historical claims are more useful later for evaluation and benchmarking than as the legal authority for a current claim.

2. Core Backend Flows

Analysis request lifecycle

Flow A — Evidence ingestion

1. Veteran authorizes a supported data source or uploads a document.
2. Backend creates a SourceDocument and records provenance.
3. Parser extracts text and structured fields; scanned pages fall back to OCR.
4. Extracted facts are normalized into EvidenceItem records.
5. Searchable chunks are embedded and indexed; each chunk retains its source document and page/record pointer.

Flow B — Claim / question analysis

1. Frontend submits a condition or review query for a case.
2. Orchestrator determines the required evidence categories and deterministic tools.
3. EvidenceService retrieves the highest-value evidence for that query.
4. RulesEngine evaluates applicable deterministic rules.
5. The LLM receives only the query, retrieved evidence, and structured rule outputs.
6. The model returns schema-constrained CaseAnalysis / ReviewItem objects.
7. Backend validates every evidenceRef and ruleResultId; invalid references fail closed.
8. Validated output becomes the VSO review payload.

Flow C — VSO review

1. Frontend requests the review brief.
2. Findings are grouped by condition/category and paired with justification and evidence.
3. The VSO selects Confirm, Reject, or Needs Review.
4. The human decision is persisted separately from the AI output for auditability and later evaluation.

Trust & provenance invariants

• Source first: every claim in the generated brief must be traceable to evidence.

• Human authority: the VSO's recorded decision is distinct from the model's suggested state.

• Deterministic separation: rules/code produce deterministic results; the model can explain them but cannot overwrite them.

• Fail closed: missing evidence, parse uncertainty, or invalid references become Needs Review—not invented certainty.

3. Backend LLD

Service contracts

CaseService — createCase(), getCase(), getCaseStatus()

EvidenceIngestionService — ingestFHIRRecords(), ingestDocument(), normalizeSource()

DocumentProcessor — extractText(), detectDocumentType(), extractStructuredFields(), chunkForRetrieval()

EvidenceService — saveEvidence(), getEvidenceById(), searchEvidence(caseId, query, filters)

RulesEngine — evaluateRule(), computePresumptiveEligibility(), evaluateApplicableRules()

AnalysisOrchestrator — analyzeCase(), retrieveRelevantEvidence(), callDeterministicTools(), validateModelOutput()

ReviewService — getCaseBrief(), recordReviewDecision(), getReviewHistory()

Core domain model

VeteranCase — id, routingId, status, timestamps

SourceDocument — caseId, sourceType, mimeType, processingStatus, provenance

EvidenceItem — sourceDocumentId, evidenceType, normalizedValue, date, page/section, sourceText, extractionConfidence

Condition — caseId, name, claimType, status

RuleResult — ruleId, result [MATCH | NO_MATCH | NOT_ENOUGH_DATA], inputs, explanation, evidenceRefs[]

CaseAnalysis — caseId, query, summary, modelVersion, createdAt

ReviewItem — category, finding, justification, evidenceRefs[], ruleResultIds[], suggestedState

ReviewDecision — reviewItemId, reviewerId, decision, note, createdAt

Frontend / backend API contract

```
POST   /api/cases                                  Create a non-PII case/routing ID
POST   /api/cases/{caseId}/documents               Upload and start processing a document
GET    /api/cases/{caseId}/documents/{id}/status
POST   /api/cases/{caseId}/analyze                 Start analysis for a query/condition
GET    /api/cases/{caseId}/review                  Return validated review payload
POST   /api/cases/{caseId}/review/{itemId}         Record VSO review decision
```

Example review item

```
{
  "id": "ri_1",
  "category": "CURRENT_CONDITION",
  "finding": "Tinnitus is documented in the medical record.",
  "suggestedState": "CONFIRM",
  "evidenceRefs": ["ev_44"],
  "ruleResultIds": []
}
```

The frontend should depend on this review contract, not on model-provider-specific output. That lets frontend and backend teams work independently against the same mock payload.

4. AI Orchestration, Security & Hackathon Plan

AI orchestration contract

Model inputs

• Veteran’s current query / condition.
• Retrieved EvidenceItem objects relevant to that query.
• Deterministic RuleResult objects.
• Strict JSON schema for CaseAnalysis and ReviewItem.

Model responsibilities

• Identify which supplied evidence is relevant.
• Explain what each source establishes.
• Organize findings into reviewable units.
• Surface ambiguity as Needs Review.

Model restrictions

• Cannot create new EvidenceItem IDs or RuleResult IDs.
• Cannot make a final claim decision, predict a rating, or fabricate missing evidence.
• Cannot replace deterministic tool outputs with its own interpretation.

Validation gate

EvidenceItem and RuleResult IDs are created before model generation. validateModelOutput() checks every returned reference against the backend-owned set. Hallucinated or unauthorized references are rejected; the result is regenerated or marked for review.

Recommended MVP stack

API: FastAPI

Database + retrieval: PostgreSQL + pgvector

File storage: StorageAdapter over local dev storage or object storage

Document parsing: native text extraction first; OCR only for scanned/image-only pages

LLM: provider-agnostic adapter with structured/schema output

Async work: background worker only if parsing latency requires it; otherwise keep the demo pollable and simple

Security / privacy assumptions

• Treat medical and service records as sensitive data.
• Frontend/session state uses a routing ID, not SSN, VA file number, or full identity.
• Do not log raw medical text, document contents, or full prompts.
• Keep raw uploads only as long as required by the demo/processing lifecycle.
• Preserve provenance, model version, and reviewer decisions for auditability.
• Production VA authorization, retention, VSO identity verification, HIPAA/FedRAMP posture, and compliance review are follow-on design work.

Hackathon build order

1. Freeze the review JSON contract and hand the mock payload to frontend.
2. Implement case creation + PDF upload.
3. Parse one or two representative document types and persist evidence with page citations.
4. Add pgvector retrieval.
5. Implement one deterministic rules example from service facts.
6. Implement schema-constrained AI orchestration + reference validation.
7. Expose GET /review and POST /review/{itemId}; connect the live frontend.
8. If time remains, add VA/FHIR sandbox/mock ingestion behind the same ingestion interface.

Out of scope

Training a predictive claims model, production VA onboarding, autonomous filing,exhaustive support for every claim type, full compliance certification, and replacing existing VSO case-management systems.